

LABORATORY

LAB 02

Fundamentals of Digital Input

Reading a Push Button and Controlling GPIO with EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	Digital Input and Logic Levels	2
2.2	Push Button as a Digital Input	2
2.3	External LED and Current-Limiting Resistor	3
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Pin Mapping	3
3.3	External LED Connection	3
4	EduFramework API	4
4.1	<code>pinMode()</code> with <code>INPUT</code>	4
4.2	<code>digitalRead()</code>	4
4.3	<code>digitalToggle()</code>	5
5	Lab Exercise	5
5.1	Requirements	5
5.2	Program	5
5.3	Verification	6
6	Extension	7
6.1	External Push Button and Pull Resistors	7
6.2	Button Debouncing	7
7	References	7

1 Introduction

1.1 Lab Overview

Digital Input allows a microcontroller to receive logic states from external signals through GPIO. When a pin is configured as a digital input, the program can read the state at that pin and use the result to perform corresponding control actions.

This lab is designed to introduce Digital Input through a push button and the EduFramework APIs. The state of the push button is used to control an external LED connected to GPIO.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Explain the basic principles of Digital Input and logic states.
2. Use EduFramework to configure and read the state of a push button through GPIO.
3. Connect and control an external LED using GPIO.
4. Build and verify an application that uses a push button to change the state of an LED.

2 Background

2.1 Digital Input and Logic Levels

GPIO (General-Purpose Input/Output) can be configured as a digital input or output. When configured as a Digital Input, a GPIO pin is used to read the logic level of an external signal.

In EduFramework, Digital Input states are represented by `LOW` and `HIGH`.

State	Value	Meaning
<code>LOW</code>	<code>0U</code>	A low logic level is detected at the Digital Input.
<code>HIGH</code>	<code>1U</code>	A high logic level is detected at the Digital Input.

Table 2.2.1. Logic states of a Digital Input

`HIGH` and `LOW` do not represent fixed voltage values. The voltage thresholds recognized as logic high or logic low depend on the electrical characteristics of the microcontroller. The limits for the S32K1xx are specified in the NXP Data Sheet [2].

2.2 Push Button as a Digital Input

A push button has two basic states: pressed and released. When operating the button changes the logic level at a Digital Input, the program can read this level to determine the current state of the button.

In a typical application, a Digital Input can be read continuously in the main loop so that the program can respond when the signal state changes.

2.3 External LED and Current-Limiting Resistor

An LED (Light-Emitting Diode) is a polarized component with an anode and a cathode. When an LED is driven from GPIO, a series resistor is used to limit the current through the LED [4] .

When connecting an external LED to the development board, ensure the correct LED polarity and use a common GND between the external circuit and the board.

3 Hardware Setup

3.1 Required Hardware

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
LED	External LED used as a visual output.
330 Ω resistor	Series resistor used to limit current through the LED.
Breadboard	Used to assemble the LED and resistor circuit.
Jumper wires	Used to connect the external circuit to the development board.

Table 2.3.1. Hardware required for the lab

3.2 Pin Mapping

The lab uses the onboard `BTN0` push button as a Digital Input and Logical Pin `GPIO0` as a Digital Output. The hardware mapping is shown in the table below [3] .

Component	Logical Pin	MCU Pin	Function
Onboard push button	<code>BTN0</code>	<code>PTC12</code>	Digital Input
External LED	<code>GPIO0</code>	<code>PTE0</code>	Digital Output

Table 2.3.2. Digital I/O pin mapping used in the lab

3.3 External LED Connection

The external LED is connected in series with a current-limiting resistor between `GPIO0` and GND. The connection is shown in the schematic below.

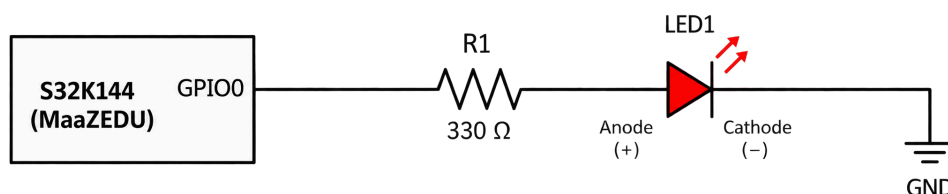


Figure 2.3.1. External LED connection to GPIO

Note

An LED is a polarized component. Identify the anode and cathode correctly before powering the circuit.

4 EduFramework API

The lab uses the EduFramework Digital I/O APIs to configure an input, read its logic state, and change the state of a Digital Output.

4.1 `pinMode()` with `INPUT`

`pinMode()` configures the operating mode of a Logical Pin. In this lab, the `INPUT` mode is used to configure a pin as a Digital Input.

`pinMode`

Configure the specified Logical Pin as a Digital Input.

Syntax

```
pinMode(pin, INPUT);
```

Parameters

Parameter	Type / accepted value	Description
<code>pin</code>	Logical Pin	Pin to be configured as a digital input.
<code>mode</code>	<code>INPUT</code>	Digital Input mode.

Return value

No return value.

Example:

```
pinMode(BTN0, INPUT);
```

4.2 `digitalRead()`

`digitalRead()` is used to read the current logic state of a Logical Pin [1].

`digitalRead`

Read the current logic state of the specified Logical Pin.

Syntax

```
digitalRead(pin);
```

Parameters

Parameter	Type / accepted value	Description
<code>pin</code>	Logical Pin	Digital Input pin whose state is read.

Return value

`HIGH` when a high logic level is detected; `LOW` when a low logic level is detected.

Example:

```
if (HIGH == digitalRead(BTN0))
{
    /* Button is pressed */
}
```

4.3 digitalToggle()

`digitalToggle()` toggles the current logic state of a Digital Output.

digitalToggle

Toggle the current logic state of the specified Logical Pin.

Syntax

```
digitalToggle(pin);
```

Parameters

Parameter	Type / accepted value	Description
pin	Logical Pin	Digital Output pin whose state is toggled.

Return value

No return value.

Example:

```
digitalToggle(GPI00);
```

5 Lab Exercise

5.1 Requirements

Develop a program that uses `BTN0` to control an external LED connected to `GPI00`. The program must satisfy the following requirements:

- The external LED is initially off.
- Each time `BTN0` is pressed, the LED state is toggled once.
- Holding `BTN0` does not cause the LED to toggle continuously.
- After `BTN0` is released, the program is ready to detect the next press.

5.2 Program

Implement the following program in `src/main.c`:

```
#include "Arduino.h"

int main(void)
{
    bool pressed = false;

    setup();

    pinMode(BTN0, INPUT);
    pinMode(GPIO0, OUTPUT);

    digitalWrite(GPIO0, LOW);

    while (1)
    {
        if ((HIGH == digitalRead(BTN0)) && (false == pressed))
        {
            digitalWrite(GPIO0);
            pressed = true;
        }

        if (LOW == digitalRead(BTN0))
        {
            pressed = false;
        }
    }

    return 0;
}
```

Listing 2.5.1. Program for controlling an external LED with a push button

`BTN0` is configured as a Digital Input and `GPIO0` as a Digital Output. The LED is initialized with `digitalWrite(GPIO0, LOW)`.

The `pressed` variable records whether the current button press has already been processed. When `BTN0` is `HIGH` and `pressed` is `false`, the program calls `digitalToggle()` to toggle the LED and sets `pressed` to `true`. Therefore, continuing to hold the button does not toggle the LED repeatedly.

When the button is released and `digitalRead(BTN0)` returns `LOW`, `pressed` is reset to `false`. The program can then process the next button press.

5.3 Verification

Build and upload the program to the MaaZEDU Development Board. Observe the external LED while pressing, holding, and releasing `BTN0`.

EXPECTED RESULT

The external LED is initially off. Each time `BTN0` is pressed, the LED changes state exactly once. Holding the button does not cause continuous toggling. After the button is released, the next press toggles the LED again.

6 Extension

6.1 External Push Button and Pull Resistors

When a Digital Input is not actively connected to a high or low logic level, the state at the pin may be undefined. A pull-up or pull-down resistor is used to establish a default input state when the switch is open.

The S32K1xx integrates pull-up and pull-down resistors for I/O pins. Under 3.3 V I/O conditions, the Data Sheet specifies an internal pull-resistor range of 20 kΩ to 60 kΩ [2].

EduFramework provides two Digital Input modes with internal pull resistors:

Mode	Function
INPUT_PULLUP	Digital Input with an internal pull-up resistor.
INPUT_PULLDOWN	Digital Input with an internal pull-down resistor.

Table 2.6.1. Digital Input modes with internal pull resistors

Example:

```
pinMode(pin, INPUT_PULLUP);  
pinMode(pin, INPUT_PULLDOWN);
```

Extension: Replace the onboard push button with an external push button connected to another GPIO Logical Pin. Select INPUT_PULLUP or INPUT_PULLDOWN according to the circuit connection and adjust the condition used to detect the pressed state.

6.2 Button Debouncing

The mechanical contacts of a push button can generate several short transitions while closing or opening. This phenomenon is called contact bounce and can cause one physical action to be detected as multiple digital events [5].

Debouncing is the process of suppressing these unwanted transitions. Depending on the application requirements, debouncing can be implemented in hardware or software.

EduFramework automatically enables the passive input filter for Logical Pins identified as onboard push buttons. This mechanism helps filter short input pulses, but it does not replace a complete debounce method in every case.

Extension: Use an external push button and propose a software debounce method that processes the button state only after the signal has remained stable for a defined period.

7 References

- [1] Arduino, *digitalRead()*, Arduino Language Reference.
- [2] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet*, S32K1XX, Rev. 15, 2026.
- [3] MaaZEDU, *MaaZEDU Development Board Guide*.
- [4] University of Wisconsin-Madison, *GPIO Pins*, ECE353 - Introduction to Microprocessor Systems.
- [5] Texas Instruments, *Debounce a Switch*, SCEA094.